MIB: A Mechanistic Interpretability Benchmark

Aaron Mueller [*1] **Atticus Geiger** [*2] **Sarah Wiegreffe** [3]
Dana Arad [4] **Iván Arcuschin** [5] **Adam Belfki** [6] **Yik Siu Chan** [7] **Jaden Fiotto-Kaufman** [6] **Tal Haklay** [4]
Michael Hanna [8] **Jing Huang** [9] **Rohan Gupta** [10] **Yaniv Nikankin** [4] **Hadas Orgad** [4] **Nikhil Prakash** [6]
Anja Reusch [4] **Aruna Sankaranarayanan** [11] **Shun Shao** [12] **Alessandro Stolfo** [13] **Martin Tutek** [4] **Amir Zur** [2]
David Bau [6] **Yonatan Belinkov** [4]

Abstract

How can we know whether new mechanistic interpretability methods achieve real improvements? In pursuit of lasting evaluation standards, we propose MIB, a Mechanistic Interpretability Benchmark, with two tracks spanning four tasks and five models. MIB favors methods that *precisely* and *concisely* recover relevant causal pathways or causal variables in neural language models. The **circuit localization track** compares methods that locate the model components—and connections between them—most important for performing a task (e.g., attribution patching or information flow routes). The **causal variable localization track** compares methods that featurize a hidden vector, e.g., sparse autoencoders (SAEs) or distributed alignment search (DAS), and align those features to a task-relevant causal variable. Using MIB, we find that attribution and mask optimization methods perform best on circuit localization. For causal variable localization, we find that the supervised DAS method performs best, while SAE features are not better than neurons, i.e., non-featurized hidden vectors. These findings illustrate that MIB enables meaningful comparisons, and increases our confidence that there has been real progress in the field.

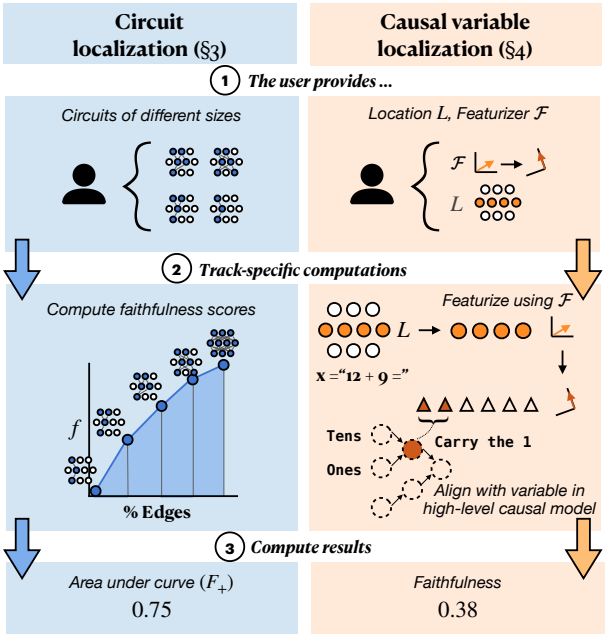

Figure 1. An overview of MIB. We compare different circuit (§3) and causal variable (§4) localization methods on their ability to faithfully represent a model's behavior on a given task. We provide standardized datasets and metrics for this purpose, and accept user submissions for display on two public leaderboards (§2).

1. Introduction

To understand how and why language models (LMs) behave the way they do, we must understand the underlying causes of their behavior. To this end, mechanistic interpretability (MI) methods have proliferated quickly. MI methods can yield deep insights into LM behaviors (Räuker et al., 2023; Ferrando et al., 2024; Sharkey et al., 2025, *i.a.*), and sometimes yield more fine-grained control over LM behaviors than standard training or inference techniques (Meng et al., 2022; Marks et al., 2025). However, it is difficult to directly compare the efficacy of MI methods. New methods are often compared to prior methods via ad hoc evaluations using metrics that may not produce generalizable insights. Thus, how can we know whether new methods are producing real advancements over prior work?

We propose a benchmark to provide a basis for comparisons.

[*]Equal contribution [1]Boston University [2]Pr(AI)[2]R Group [3]Ai2 [4]Technion [5]University of Buenos Aires [6]Northeastern University [7]Brown University [8]University of Amsterdam [9]Stanford University [10]Independent [11]MIT [12]Cambridge University [13]ETH Zürich. Correspondence to: Aaron Mueller <amueller@bu.edu>.

Proceedings of the 42^{nd} International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025.

A benchmark is a claim as to what should be considered important to a field. In our case, *the ability to precisely locate, and causally validate, task mechanisms or specific concepts in a neural network* is the key goal of (at least some part of) many MI pipelines. In fact, some have argued that causal analysis and localization are what differentiate MI from other types of interpretability work (Mueller et al., 2024; Geiger et al., 2024a; Saphra & Wiegreffe, 2024). Existing benchmarks compare *within* a specific class of methods (Karvonen et al., 2025; Schwettmann et al., 2023), or on specific tasks and models (Arora et al., 2024; Huang et al., 2024a; Miller et al., 2024; Gupta et al., 2024).

We propose MIB to encourage stable standards for comparing *across* MI methods—specifically, localization and featurization methods—in a principled way. MIB encourages evaluation across a standard suite of models, datasets with fixed counterfactual inputs used for interventions (§2), and principled metric definitions—including novel metrics (§3.1). It includes two public leaderboards that accept submissions for evaluation on a private test set (§2.4).

MIB contains two tracks based on two prominent paradigms in mechanistic interpretability: **circuit localization** (§3) and **causal variable localization** (§4). The circuit localization track benchmarks how well methods can locate the most important subset of model components for performing a given task (Cao et al., 2020; Wang et al., 2023; Conmy et al., 2023). The causal variable localization track benchmarks methods for featurizing hidden vectors (e.g., mapping them to an alternative vector space) and selecting features that implement specific causal variables or concepts (Vig et al., 2020; Geiger et al., 2021; 2024a; Mueller et al., 2024).

Beyond standardizing evaluations, MIB yields several scientific insights. For instance, using MIB, we find that attribution and mask optimization methods outperform other approaches to circuit localization. We find that supervised methods provide better features for causal variable localization, but the popular method of sparse autoencoders (SAEs) fail to provide better features that standard dimensions of hidden vectors. This is evidence that (1) there is clear differentiation between methods, and (2) there has been real progress in mechanistic interpretability.

Our dataset is available at this HuggingFace URL. Code is available at this GitHub URL. The leaderboard is hosted at this HuggingFace URL.

2. Materials

2.1. Tasks

Both tracks evaluate across four tasks. The tasks are selected to represent various reasoning types, difficulty levels, and answer formats. Two of the tasks—Indirect Object Identification and Arithmetic—were chosen because they have been extensively studied, while the others—Multiple-choice Question Answering and the AI2 Reasoning Challenge—were chosen precisely because they have *not* been studied.[1]

The number of instances in each dataset and split is summarized in Table 5 (App. C). Each task comes with a training split on which users can discover circuits or causal variables, and a validation split on which users can tune their methods or hyperparameters. We also create two test sets per task: public and private. The public test set enables faster iteration on methods. We release the train, validation, and public test sets on Huggingface. The private test set is not visible to users; they must upload either their circuits or their locations and featurizers to an API (see §2.4).

Indirect Object Identification (IOI). The indirect object identification (IOI) task, first proposed by Wang et al. (2023), is one of the most studied tasks in MI. IOI has sentences like *"When Mary and John went to the store, John gave an apple to _"*, containing a subject (*"John"*) and an indirect object (*"Mary"*), which should be completed with the indirect object. Even small LMs can achieve high accuracy; thus, it has been well studied (Huben et al., 2024; Conmy et al., 2023; Merullo et al., 2024). We generate 40,000 instances. We ensure that each name tokenizes to a single token for all models we test. To assess generalization, the private test set contains names and direct objects that are not contained in the public train or test set. See App. C.1 for details.

Arithmetic. Math-related tasks are common in MI (Stolfo et al., 2023; Nanda et al., 2023; Zhang et al., 2024; Nikankin et al., 2025) and interpretability research more broadly (Liu et al., 2023; Huang et al., 2024b). We follow Stolfo et al. in defining the task as performing operations with two operands of up to two digits each. Given a pair of numbers and an operator, the model must predict the result of the operation, e.g., *"What is the sum of 13 and 25?"*. To create the dataset, we enumerate all possible pairs of one-digit and two-digit numbers and generate queries for addition and subtraction, yielding about 75,000 instances. Following Karpas et al. (2022) and Stolfo et al. (2023), we use six natural language templates for each operand pair to ensure we are isolating robust behavior. See App. C.2 for details.

Multiple-choice question answering (MCQA). MCQA is a common task format on LM evaluation benchmarks, though only a few MI works have studied it (Lieberum et al., 2023a; Wiegreffe et al., 2025; Li & Gao, 2024). We expand an existing synthetic dataset designed to isolate a model's MCQA ability from any task-specific knowledge (Wiegreffe

[1] Studying the same tasks can lead to hill-climbing on narrow distributions. Insights from novel models and task settings could verify that previous advancements are real.

et al., 2025); the information needed to answer the questions is contained in the prompt. We generate 260 instances. All questions have four choices and are about the color of an object, such as:

```
Question: A box is brown. What color is a box?
A. gray
B. black
C. white
D. brown
Answer: D
```

Table 1. Model performance (0-shot, greedy generation) for all models and tasks on our public test splits. For results using ranked-choice scoring and more details on evaluation, see App. C.5.

			Arithmetic		ARC	
	IOI	MCQA	(+)	(−)	(E)	(C)
Llama-3.1 8B	0.71	0.92	0.96	0.88	0.93	0.79
Gemma-2 2B	0.83	1.00	0.65	0.43	0.79	0.59
Qwen-2.5 0.5B	0.99	1.00	0.37	0.29	0.73	0.58
GPT2-Small	0.92	0.06	0.00	0.10	0.03	0.03

AI2 Reasoning Challenge (ARC). Finally, we analyze the ARC dataset (Clark et al., 2018), which comprises grade-school-level multiple-choice science questions. This is a representative task for evaluating basic scientific knowledge in LMs (Brown et al., 2020; Jiang et al., 2023; Dubey et al., 2024). Our work presents the first causal investigation of LM performance on this dataset, and among the first MI investigations of a more realistic task used to benchmark state-of-the-art LMs. We follow the dataset's original partition to Easy and Challenge subsets (5000 and 2500 instances, respectively) and analyze them separately; this is due to a large accuracy difference on the two subsets (Table 1). We maintain the original 4-choice multiple-choice prompt formatting (see App. C.4), making this dataset related in format to, but more challenging than, MCQA.

2.2. Counterfactual Inputs

For both MIB tracks, counterfactual interventions on model components[2] provide the basis for all evaluations. Here, components are set to the value they would have taken if a *counterfactual input* were provided. *Activation patching* is a popular term for this method.[3] Several studies (Vig et al., 2020; Geiger et al., 2021; Chan et al., 2022) have argued that this type of intervention is a useful analysis tool because components are only ever fixed to values that they would *actually realize* (as opposed to interventions that add noise or fix components to constants).

In the circuit localization track, activation patching is used to push models towards answering in an opposite manner to how they would naturally answer given the input. Success is achieved in this setting when counterfactual interventions to components outside the circuit minimally change the model's predictions. In the causal variable localization track, activation patching is used to precisely manipulate specific concepts. Success is achieved in this setting when a variable in a causal model is a faithful summary of the role a model component plays in input-output behavior—i.e., interventions on the variable have the same effect as interventions on the model component.

The counterfactual input defines the task to a large extent (Miller et al., 2024);[4] thus, it is crucial that the mapping from a dataset instance to its counterfactual counterpart is fixed. For each task, we define a set of meaningful counterfactual inputs to help localize model behaviors. Some of these maintain the same correct answers as the original instances; some do not. For example, the counterfactual inputs for ARC and MCQA have different answer symbols or answer orders than the original instance, which change the correct answer. Others change the semantics of the input, which may or may not change the correct answer.

We provide counterfactual inputs for each instance in the train, validation, and test sets, where the mappings from the original inputs to the counterfactual inputs are fixed to ensure consistency in evaluation. These are provided on Huggingface. See App. C, Tables 6, 8, 9, and 10 for examples and App. C for more details.

2.3. Models

To provide baselines for our paper and to initialize entries in the public leaderboard, we evaluate a set of open-weight models. Given the pace of the field, any set of models or tasks will be incomplete; we select 4 models that cover a range of model sizes, families, capability levels, and prominence in MI: Llama-3.1 8B (Dubey et al., 2024), Gemma-2 2B (Riviere et al., 2024), Qwen-2.5 0.5B (Yang et al., 2024), and GPT-2 Small (117M, Radford et al., 2019).

For a mechanistic analysis to be meaningful for a given

[2]We use "component" as a generic term to refer to any (part of a) hidden representation in a model. When referring to submodules like full MLP blocks, we refer to the output vectors of these blocks.

[3]The term *activation patching* includes not only interventions from counterfactual inputs, but also interventions that zero out activations, inject noise, or steer activations to off-distribution values (Wang et al., 2023; Zhang & Nanda, 2024); we use this term because we use mean ablations and optimal ablations as baselines in §3.2. Terms like *resampling ablations* (Chan et al., 2022) or *interchange interventions* (Geiger et al., 2021) more narrowly refer to interventions from counterfactual inputs.

[4]For example, given IOI, if the counterfactual entails replacing a name with a randomly selected one, then the task is now to choose the correct indirect object over a random name; this is distinct from simply generating the correct indirect object.